

Accio

A Rust Concepts Study Guide

This guide covers every Rust concept encountered while building **Accio**, a recon and OSINT multitool. It is written to be read *after* the code already works, so that the patterns you typed become patterns you understand.

Covered: statements and expressions · structs and enums · pattern matching · Result and error handling · the turbofish · ownership, borrowing and cloning · modules and visibility · closures · futures and async/await · the tokio runtime · spawning and concurrency · clap · terminal colour · git workflow.

How to use this: read a section, then open the matching file in `src/` and find the lines it describes. The review questions in section 14 are the fastest way to find the gaps.

1 · Statements, Expressions, and the Missing Semicolon

Rust draws a sharp line between a **statement** (does something, produces no value) and an **expression** (evaluates to a value). A semicolon turns an expression into a statement by discarding its value. Leaving the semicolon off the **last** expression in a block makes that value the value of the whole block.

```
fn add(a: i32, b: i32) -> i32 {
    a + b          // no semicolon: this IS the return value
}

fn add_wrong(a: i32, b: i32) -> i32 {
    a + b;         // semicolon: value discarded, function returns ()
}                  // ERROR: expected i32, found ()
```

This one rule explains three separate things that came up while building Accio.

Match arms return values

```
match timeout(...).await {
    Ok(Ok(_)) => PortStatus::Open,    // no semicolon
    Ok(Err(_)) => PortStatus::Closed, // each arm produces a value
    Err(_)    => PortStatus::Timeout,
}
```

Each arm is an expression. The whole match evaluates to whichever arm matched, and because it is the last thing in the function, that becomes the return value — no return keyword needed.

Returning a Vec at the end of a function

```
pub async fn full_port_scan(ip: String) -> Vec<u16> {
    let mut open_ports = Vec::new();
    // ... loop that pushes into open_ports ...
    open_ports          // no semicolon: this is the return value
}
```

Why a for loop can silently break your return type

A for loop always evaluates to `()` — it never produces a value. If a return expression accidentally ends up *inside* the loop's braces, the loop becomes the last expression in the function, and the compiler reports `expected Vec<u16>, found ()`.

```
// BROKEN — open_ports is trapped inside the loop
for handle in handles {
    if let Ok((port, status)) = handle.await { ... }
    return open_ports // still inside the for loop
}

// CORRECT — the loop closes first
for handle in handles {
    if let Ok((port, status)) = handle.await { ... }
}
open_ports // now the function's final expression
```

Debugging tip: when an error says *expected T, found ()*, suspect brace placement or a stray semicolon before suspecting your logic.

2 · Structs and Enums

Structs — one thing with several named parts

A struct groups related fields into a single type. Fields are separated by **commas**, never semicolons. There are two struct shapes, and mixing them is a common early mistake.

```
struct Cli { command: Commands } // named-field: braces, named fields
struct Point(i32, i32); // tuple struct: parens, no field names

struct Cli() { command: Commands; } // ERROR: mixes both shapes
```

Enums — one thing that is exactly one of several possibilities

An enum value is exactly one variant at a time. This is far more powerful in Rust than in C, because variants can carry data.

```
pub enum PortStatus { // simple: variants carry no data
    Open, Closed, Timeout, InvalidAddress,
}

enum Commands { // struct-like: variants carry named fields
    Scan { p: u16, ip: String },
    Fullscan { ip: String },
}
```

PortStatus answers "what happened to this port?" with one of four answers. Commands answers "what did the user ask for?" and carries the arguments along with the answer.

Why an enum beat a bool

The first scanner returned `bool`. That works for two outcomes, but a port that times out is genuinely different from one that refuses the connection — a timeout usually means a firewall is dropping packets, while a refusal means the host is reachable but nothing is listening. A `bool` cannot express that; an enum can, and the compiler then forces every caller to handle each case.

Design principle: when a value has more than two meaningful states, reach for an enum rather than overloading a `bool` or reusing an existing variant to mean two different things.

3 · Pattern Matching: match, if let, Destructuring

match must be exhaustive

Rust requires that a match handle every variant. This is a feature: when `InvalidAddress` was added to `PortStatus`, the compiler immediately pointed at every place needing an update.

```
match port_scanner(ip, p).await {
    PortStatus::Open => println!("Port {p} => OPEN"),
    PortStatus::Timeout => println!("Port {p} => TIMEOUT"),
```

```

    PortStatus::Closed          => println!("Port {p} => CLOSED"),
    PortStatus::InvalidAddress => println!("Invalid IP Address Format"),
}

```

Destructuring pulls data out of variants

```

match cli.command {
    Commands::Scan { p, ip } => { /* p and ip are now local variables */ }
    Commands::Fullscan { ip } => { /* ip is available here */ }
}

```

Same mechanism as `Ok(value)` — tuple-style variants destructure with parentheses, struct-style variants with braces.

if let — a match that only cares about one case

```

match handle.await { // long form
    Ok((port, status)) => { /* use them */ }
    Err(_) => { /* nothing to do */ }
}

if let Ok((port, status)) = handle.await { /* use them */ } // identical

```

Nesting works the same way, which is how Accio filters for open ports:

```

if let Ok((port, status)) = handle.await {
    if let PortStatus::Open = status {
        open_ports.push(port);
    }
}

```

Result helper methods

Sometimes even `if let` is more than needed. Use these when you only need to know *whether* it worked, not what the value was.

```

address.parse::<SocketAddr>().is_err() // true if parsing failed
address.parse::<SocketAddr>().is_ok()  // true if parsing succeeded

if address.parse::<SocketAddr>().is_err() {
    return PortStatus::InvalidAddress;
}

```

4 · Result and Honest Error Handling

`Result<T, E>` is an enum with two variants: `Ok(T)` holding a success value, `Err(E)` holding an error. Rust has no exceptions; fallible operations return a `Result` and the caller must deal with it.

unwrap() and why it is a landmine

```

let addr = format!("{}", ip, port).parse::<SocketAddr>().unwrap();

```

`unwrap()` means "give me the value inside, and if this is an `Err`, crash the program." Fine in a throwaway example with hardcoded input. Not fine the moment input comes from a user, because one malformed argument takes down the whole tool with a panic instead of a helpful message.

The three honest alternatives

```

// 1. Handle both cases explicitly
let addr: SocketAddr = match address.parse() {
    Ok(a) => a,
    Err(_) => return PortStatus::InvalidAddress,
};

// 2. Check without extracting
if address.parse::<SocketAddr>().is_err() { return PortStatus::InvalidAddress; }

// 3. Ignore only the case you truly do not care about
if let Ok(a) = address.parse::<SocketAddr>() { /* ... */ }

```

Nested Results

Wrapping one fallible operation in another produces a Result inside a Result — exactly what happens when a connection attempt is wrapped in a timeout.

```
match timeout(Duration::from_millis(500), TcpStream::connect(&addr)).await {
    Ok(Ok(_)) => PortStatus::Open,      // finished in time AND connected
    Ok(Err(_)) => PortStatus::Closed,    // finished in time BUT refused
    Err(_)    => PortStatus::Timeout,    // did not finish in time at all
}
```

Read the outer layer first: did the timeout expire? Then the inner: did the operation itself succeed? Each layer answers a different question, and collapsing them into one would lose information.

5 · Type Paths and the Turbofish

`::` is the path separator. It navigates into types, modules, and enums.

```
PortStatus::Open      // variant of an enum
TcpStream::connect(...) // function belonging to a type
Duration::from_millis(500) // constructor belonging to a type
std::net::SocketAddr  // module path to a type
```

The turbofish: `::`

Some methods are generic — they can produce many different types and cannot know which you want. `parse()` is the classic case: it can turn a string into an integer, a float, an IP address, or a socket address. Rust needs the target type from somewhere, and there are two ways to supply it.

```
let addr: SocketAddr = address.parse().unwrap(); // inferred from the variable
address.parse::<SocketAddr>().is_err()           // stated inline (turbofish)
```

The second form is *required* when there is no variable to annotate — as in a condition that immediately calls `.is_err()`. The name comes from the shape: `::<>` looks like a small fish.

6 · Ownership, Borrowing, and Cloning

Ownership is what makes Rust memory-safe without a garbage collector. Three rules cover almost everything:

- 1 Every value has exactly one owner.
- 2 When the owner goes out of scope, the value is dropped (freed).
- 3 A reference (borrow) can never outlive the value it points to.

Moving vs borrowing

```
let ip = String::from("127.0.0.1");
port_scanner(ip, 80).await; // ownership MOVES into port_scanner
println!("{}", ip);         // ERROR: value borrowed after move

let ip = String::from("127.0.0.1");
port_scanner(&ip, 80);       // BORROWS – ip is only lent out
println!("{}", ip);         // fine: ip is still owned here
```

Why the signature changed from `&str`; to `String`

The original synchronous scanner took `ip: &str`, and that was correct: the function glanced at the string, built an address, and finished. Borrowing avoided copying anything.

Spawned async tasks break that assumption. A task handed to `tokio::spawn` may run at any later point — possibly after the loop iteration that created it has ended and its local variables are gone. A borrow into that scope would point at freed memory, which is exactly the use-after-free bug Rust exists to prevent. The compiler refuses it, so each task must **own** its data.

```
pub async fn port_scanner(ip: String, port: u16) -> PortStatus
```

Cloning

If 65,535 tasks each need to own an IP string, each needs its own copy. `.clone()` makes a genuine, independent duplicate.

```
for port in 1..=65535 {
    let ip_clone = ip.clone();           // one private copy per task
    let handle = tokio::spawn(async move {
        let status = port_scanner(ip_clone, port).await;
        (port, status)
    });
    handles.push(handle);
}
```

`port` needed no clone. Small types like `u16` implement `Copy`: they live on the stack and duplicate automatically. `String` owns heap memory, so duplication has to be explicit.

Rule of thumb: borrow (`&str`) when data is read briefly and the owner outlives the use. Own (`String`) when data must travel — across threads, into spawned tasks, or anywhere it might outlive the current scope.

7 · Modules: Splitting Code Across Files

```
src/
■■■ main.rs           - CLI definition, async main, dispatch
■■■ logo.rs           - the banner
■■■ port_scanner.rs   - single-port async scan + PortStatus
■■■ full_portscan.rs - concurrent all-ports scan
```

Declaring and importing are two different steps

Declaring a module tells the compiler the file exists. **Importing** brings a specific item into scope.

```
// in main.rs - declare each module exactly once, at the crate root
mod logo;
mod port_scanner;
mod full_portscan;

use logo::main_logo;
use port_scanner::{port_scanner, PortStatus};
use full_portscan::full_portscan;
```

pub — visibility

Everything is private by default. An item must be marked `pub` to be usable outside its own module.

```
pub fn main_logo() { ... }
pub enum PortStatus { ... }
pub async fn port_scanner(...) -> PortStatus
```

crate:: — importing between sibling files

A non-root file cannot say `port_scanner::PortStatus` directly, because it is not the file that declared the module. It must start from the crate root.

```
// in full_portscan.rs
use crate::port_scanner::{port_scanner, PortStatus};
```

Common mistake: writing `mod port_scanner;` again in a sibling file. That declares a *second, separate* module rather than referring to the existing one. Declare once in `main.rs`; refer to it elsewhere with `crate::`.

8 · Closures

A closure is an anonymous function defined inline that can capture variables from the surrounding scope. A plain `fn` cannot do that — it only sees its own parameters.

```
fn add_one(x: i32) -> i32 { x + 1 }    // a normal function
```

```
let add_one = |x: i32| x + 1;           // the same thing as a closure

let name = String::from("Logan");
let greet = || println!("Hello, {}", name); // name captured by reference
greet();
```

move — capture by ownership

By default a closure borrows what it captures. `move` forces it to take ownership instead. Same ownership question as before, applied to closures: if the closure might run later, or on another thread, borrowing is unsafe.

```
let name = String::from("Logan");
let greet = move || println!("Hello, {}", name); // name moved in
greet();
// println!("{name}"); // ERROR: name no longer owned here
```

async move blocks

An `async move { ... }` block is a closure-like construct that produces a future and takes ownership of everything it captures — precisely what a spawned task needs.

```
let handle = tokio::spawn(async move {
    let status = port_scanner(ip_clone, port).await;
    (port, status) // the future's output value
});
```

9 · Async, Futures, and the Tokio Runtime

The problem

Synchronous code blocks. When `TcpStream::connect_timeout` is called, the entire program stops until it resolves. Scanning 65,535 ports sequentially, each waiting up to 500 ms, is roughly nine hours worst case — and almost all of it is spent waiting, not working.

What a future actually is

A future is a value representing work that has not finished — and, importantly, has not even *started*. Calling an `async` function does not run its body; it constructs a future and hands it back immediately.

```
let fut = TcpStream::connect(&address); // nothing has happened yet
let res = fut.await;                   // NOW the work runs
```

The buzzer analogy: a synchronous call is standing at the counter staring at the kitchen until your food arrives — nobody behind you can order. An `async` call hands you a buzzer and lets you sit down. Holding the buzzer is holding a future; `.await` is waiting for it to go off.

This separation between *describing* work and *running* it is what makes concurrency possible. Thousands of futures can exist at once, costing almost nothing, and the runtime interleaves their progress.

The tokio runtime

Rust's standard library defines what a future is but includes no executor to drive one. Tokio provides that runtime. Without the macro, `async fn main()` does not compile — there would be nothing to drive the future.

```
#[tokio::main]
async fn main() {
    // main can now be async and can use .await
}
```

Tokio's `TcpStream` is a different type

```
use std::net::TcpStream; // synchronous – blocks the thread
use tokio::net::TcpStream; // asynchronous – yields instead of blocking
```

No built-in `connect_timeout`. The synchronous type has one; tokio's does not. Instead, wrap the connection future in `tokio::time::timeout`, which races it against a clock and produces the nested `Result` from section 4.

More flexible arguments. Tokio's `connect` accepts anything implementing `ToSocketAddrs` — including a plain `&str` — and parses internally. That is why explicit `SocketAddr` parsing became unnecessary for *connecting*, and was kept only as an explicit *validation* step so malformed input could be reported distinctly.

10 · Spawning Tasks: The Two-Pass Pattern

Awaiting each future immediately is still sequential — it just uses `async` syntax:

```
for port in 1..=65535 {
    let status = port_scanner(ip.clone(), port).await;    // waits here, every time
}
```

Real concurrency means handing work to the runtime *without* waiting, then collecting results afterwards.

Pass 1 — spawn everything, wait for nothing

```
let mut handles: Vec<JoinHandle<(u16, PortStatus)>> = Vec::new();

for port in 1..=65535 {
    let ip_clone = ip.clone();
    let handle = tokio::spawn(async move {
        let status = port_scanner(ip_clone, port).await;
        (port, status)
    });
    handles.push(handle);    // collect the receipt, move on immediately
}
```

`tokio::spawn` hands the future to the runtime and returns a `JoinHandle` straight away. The loop never blocks, so all 65,535 attempts end up in flight together.

Pass 2 — collect the results

```
let mut open_ports = Vec::new();

for handle in handles {
    if let Ok((port, status)) = handle.await {
        if let PortStatus::Open = status {
            open_ports.push(port);
        }
    }
}

open_ports
```

Why awaiting a `JoinHandle` gives a Result

A spawned task runs semi-independently, so it can fail in a way the inner logic never anticipated — most commonly by panicking. Awaiting the handle yields `Result<T, JoinError>`:

<code>Ok((port, status))</code>	Task completed normally; here is its output.
<code>Err(join_error)</code>	The task itself failed — typically a panic.

Reading the `JoinHandle` type

```
JoinHandle<(u16, PortStatus)>    // correct: ONE parameter, a tuple
JoinHandle<u16, PortStatus>      // wrong: JoinHandle takes one type parameter
```

`JoinHandle<T>` is generic over a single type: whatever the task returns. Since the task returns the tuple `(port, status)`, the type is the tuple type `(u16, PortStatus)`, written with parentheses.

Where the filtering happens

`full_port_scan` returns `Vec<u16>`, not `Vec<PortStatus>`. Filtering happens inside the function, so by the time the caller receives the vector, every entry is already known to be open. There is no status left to check in `main` — just numbers to print.

```
Commands::Fullscan { ip } => {
    let open_ports = full_port_scan(ip).await;
```

```

    for port in open_ports {
        println!("Port {port} => OPEN");
    }
}

```

Design principle: let a function return the smallest, most digested result its callers actually need. Filtering once, at the source, beats making every caller re-derive the same conclusion.

11 · Building a CLI with clap

Two models exist for command-line programs. Accio uses the first, which needs no loop: `main` runs top to bottom — print banner, parse arguments, dispatch, exit.

Command-based	One invocation does one job, then exits. <code>accio scan 80 127.0.0.1</code> . Used by <code>nmap</code> , <code>git</code> , <code>RustScan</code> .
----------------------	--

Interactive shell	Launch once, type commands at a prompt in a loop. Used by <code>msfconsole</code> .
--------------------------	---

The derive pattern

```

use clap::{Parser, Subcommand};

#[derive(Parser)]
#[command(name = "accio")]
struct Cli {
    #[command(subcommand)]
    command: Commands,
}

#[derive(Subcommand)]
enum Commands {
    Scan { p: u16, ip: String },
    Fullscan { ip: String },
}

```

`#[derive(Parser)]` generates, at compile time, all the argument-parsing code: reading `std::env::args()`, validating types, producing help text, reporting errors. `Cli::parse()` calls that generated code.

Positional vs named arguments

```

accio scan 80 127.0.0.1      // positional: p then ip, matching field order
accio scan -p 80 --ip ...    // would require #[arg(long)] on each field

```

Field order in the enum variant determines argument order on the command line. Reordering the fields silently changes the CLI.

Running during development

```

cargo run -- scan 80 127.0.0.1
cargo run -- fullscan 127.0.0.1

```

The bare `--` separates cargo's own flags from the program's arguments; everything after it passes through untouched. Running `accio` directly only works once the binary is on `PATH` — during development it lives at `./target/debug/accio`.

Where the banner goes

Because `Cli::parse()` exits immediately on invalid input, the banner must print *before* parsing if it should appear on every run.

```

#[tokio::main]
async fn main() {
    main_logo();           // always runs first
    let cli = Cli::parse(); // may exit here on bad input
    match cli.command { ... }
}

```



```
}
```

RustScan uses the same ordering, and additionally gates the banner behind flags such as `--no-banner` and `--greppable` — worth adding to Accio, since banner output pollutes results another program might parse.

12 · Terminal Colour and Output

Raw ANSI escape codes

```
println!("\x1b[35mmagenta text\x1b[0m");
```

`\x1b` is the escape character; `[35m` selects magenta; `[0m` resets. Without the reset, every later line inherits the colour. Codes 31–37 cover red, green, yellow, blue, magenta, cyan, white.

The colorful crate

Crates wrap those codes in a readable API. The `Colorful` trait attaches methods directly to string types, which is why it must be imported even though it is never named.

```
use colorful::{Color, Colorful};

println!("{}", logo.gradient(Color::SeaGreen3).bold());
```

format! vs println!

`println!` builds a string and prints it, exposing nothing. `format!` builds the same string and returns it as a `String`. Styling methods need a value to act on, so interpolated text must be built first.

```
// Cannot chain .bold() onto println! – there is no value to chain onto
println!("{}", format!("Scanning {ip} on port {p}").bold());
```

Rule: `println!` to print and forget. `format!` whenever the string must be styled, stored, returned, or passed onward.

Finding a crate's real API

Guessing enum variants wastes time. Either let the compiler suggest corrections (it lists near-matches for unknown variants), or read the generated docs for the exact installed version:

```
cargo doc --open -p colorful
```

13 · Git and GitHub Workflow

First-time setup

```
echo "/target" > .gitignore      # never commit build artifacts
git init
git add .
git status                       # verify target/ is absent before committing
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/<user>/accio.git
git push -u origin main
```

`/target` holds compiled output — large, machine-specific, fully regenerable from source. It never belongs in version control.

Every push after the first

```
git add .
git commit -m "Describe what changed"
git push
```

The `-u origin main` flag is only needed once; it links the local branch to the remote so later pushes need no arguments.

The email privacy rejection

GitHub rejects pushes whose commits carry a private email address (GH007). Commit under GitHub's no-reply address instead, then rewrite the commit that already carries the wrong one:

```
git config user.email "ID+username@users.noreply.github.com"
git commit --amend --reset-author --no-edit
git push
```

14 · Review Questions

Answer these from memory before checking the guide. Any uncertain answer marks a section worth rereading.

1. Why does a for loop cause "expected Vec<u16>, found ()" when a return value ends up inside its braces?
2. What is the difference between a tuple struct and a named-field struct?
3. Why was PortStatus a better return type than bool?
4. What does the compiler do when a new variant is added to an enum that is matched somewhere?
5. When is `if let` preferable to `match`, and when is `.is_err()` preferable to `if let`?
6. In `Ok(Err(_))`, what does each layer represent?
7. Why does `.parse::<SocketAddr>().is_err()` need the `turbofish` when `let addr: SocketAddr = address.parse()` does not?
8. Why must a spawned task own its data instead of borrowing it?
9. Why does `port` need no `.clone()` while `ip` does?
10. What is the difference between declaring a module (`mod`) and importing an item (`use`)?
11. Why does a sibling file need `crate::` while `main.rs` does not?
12. What does a closure do that a plain `fn` cannot?
13. What does calling an `async` function return, and when does its body actually run?
14. What does `#[tokio::main]` provide, and why is it required?
15. Why does `tokio`'s `TcpStream` have no `connect_timeout`, and what replaces it?
16. Why does `awaiting` inside the `spawn` loop defeat the purpose of spawning?
17. Why does `awaiting` a `JoinHandle` produce a `Result`?
18. Why is `JoinHandle<(u16, PortStatus)>` correct and `JoinHandle<u16, PortStatus>` not?
19. Why does `main.rs` not check `PortStatus` when printing `fullscan` results?
20. Why must the banner print before `Cli::parse()` rather than after?

Where Accio goes next

Roughly in order of difficulty: a `--no-banner` or `--quiet` flag so output can be piped; configurable timeout and port range as CLI flags; a concurrency limit (spawning 65,535 tasks at once can exhaust file descriptors — `RustScan` solves this with batching tied to the system's `ulimit`); banner grabbing on discovered open ports; then the OSINT modules that motivated the project — DNS enumeration, certificate-transparency subdomain lookup, and `whois`.